

OpenVeinNet: A Novel Open-Set Finger Vein Verification Network Using Multi-Graph Learning and Centroid-Angular Hybrid Loss

Sushrut Patwardhan and Raghavendra Ramachandra

Abstract—Biometric verification systems play a crucial role in secure access control applications. Among the various biometric modalities, finger vein recognition has gained significant attention due to its high reliability and robustness. A key advantage of finger vein biometrics is that the vascular patterns lie beneath the skin, making them invisible to the naked eye and inherently resistant to presentation attacks. In this paper, we present OpenVeinNet, a novel open-set finger vein verification framework that integrates dynamic snake convolution with a graph convolutional network. The dynamic snake convolution effectively captures the tubular structures of finger vein patterns, while the graph convolutional network encodes these structures into discriminative feature representations for reliable verification. To further enhance performance in open-set scenarios, we introduce a *centroid-angular hybrid loss*, which jointly optimises intra-class compactness and inter-class separability in the learned feature space. Extensive experiments were conducted on five publicly available datasets including FV-300, MMCBNU, POLY-U, FV-USM, and VERA, and the proposed method was benchmarked against seven state-of-the-art approaches, including both handcrafted and deep learning methods. The results demonstrate that OpenVeinNet consistently outperforms existing techniques, achieving superior accuracy and robustness in open-set verification scenarios.

Index Terms—Biometrics, Finger vein, Deep learning, Graph Neural Network, Verification.

1 INTRODUCTION

Reliable user verification is essential for secure access control systems across a wide range of applications. Biometric authentication has emerged as a preferred solution owing to its convenience and ability to uniquely identify individuals. Conventional biometric traits such as fingerprints, facial features, irises, palmprints, and ears are widely used; however, they can be susceptible to spoofing, particularly when acquired in a non-intrusive manner. In contrast, finger vein patterns present a compelling alternative, as they are located beneath the skin and are inherently difficult to replicate or observe externally. This intrinsic resistance to presentation attacks, coupled with high verification accuracy and relatively low deployment cost, has driven the increasing adoption of finger vein recognition, especially in banking and financial services.

Finger vein patterns can be acquired using cost-effective near-infrared (NIR) imaging systems, in which the light source and camera are positioned above, alongside, or around the finger. When NIR light penetrates the finger, it is absorbed by the haemoglobin present in the veins, causing the vascular structures to appear darker than the surrounding tissue. Finger vein biometrics exhibit high temporal stability, are less affected by environmental variations, and demonstrate strong uniqueness across individuals. Furthermore, capturing images from both the dorsal and palmar sides of the finger can enhance verification accuracy.

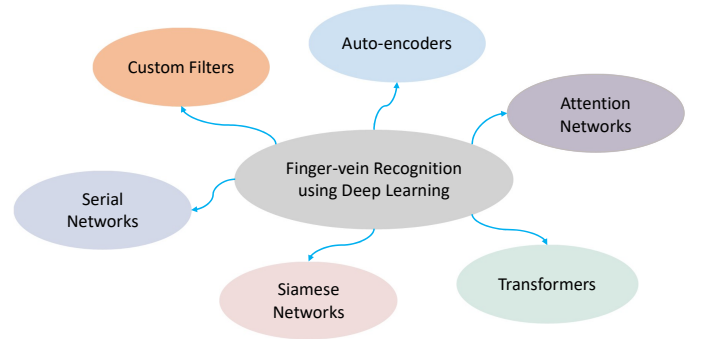


Fig. 1: A brief taxonomy of finger vein verification techniques based on deep learning approaches.

Finger vein verification has been extensively investigated in the academic literature, leading to a wide range of methods that employ both handcrafted and deep learning approaches. Earlier studies on finger vein recognition primarily relied on image processing techniques to extract vein patterns for template-based matching. Notable handcrafted feature extraction methods include Maximum Curvature Points (MCP) [34], Repeated Line Tracking (RLT) [35], Wide Line Detectors (WLD) [36], and Mean Curvature (MC), which are effective in enhancing vascular structures and achieving reliable verification accuracy under controlled acquisition conditions. In addition, various texture descriptors such as Local Binary Pattern (LBP) [37], Local Line Binary Pattern (LLBP) [38], Histogram of Oriented Gradients (HoG) [39], and Local Directional Pattern (LDP) have been widely explored. Dimensionality reduction methods,

Both authors have contributed equally.

Corresponding Author: Raghavendra Ramachandra (E-Mail: raghavendra.ramachandra@ntnu.no)

Authors (Year)	Technique Description
Huafeng Qin et al., [1] (2015)	3 Sequential convolution layers and 2 dense layers
Itqan et al., [2] (2016)	3 Sequential convolution layers and 1 dense layers
Syafeeza Radzi et al., [3] (2016)	2 Sequential convolution layers
Huafeng Qin et al., [4] (2017)	4 Sequential convolution layers with path-based training.
Hyung Gil Hong et al., [5] (2017)	12 Sequential convolution layers and 3 dense layers
Rig Das et al., [6] (2018)	5 Sequential convolution layers
Cihui Xie et al., [7] (2019)	Siamese network with 5 conventional layers trained on triplet loss function.
Jong Min Song et al., [8] (2019)	8 Sequential convolution layers trained using composite finger vein image generated by converting the 1-channel input image to a 3-channel input image.
Su Tang et al., [9] (2019)	Siamese network with residual CNN architecture.
Borui Hou et al., [10] (2019)	Autoencoder architecture.
Wenming Yang et al., [11] (2019)	Using GAN for generating enhanced images and performing verification using encoder features of GAN.
Junying Zeng et al., [12] (2020)	Deformable convolution in U-NET type architecture.
Ridvan Salih Kuzu et al., [13] (2020)	6 Sequential convolution layers and 2 dense layers and LSTM for classification.
Zhiang Hao et al., [14] (2020)	Multitask Learning based on Resnet-18 like model with SoftL1Loss and ArcFace loss
Hengyi Ren et al., [15] (2021)	Squeeze and Excitation blocks embedded Resnet trained on encrypted finger vein images.
Ridvan Salih Kuzu et al., [16] (2021)	Custom DenseNet 161 trained on additive angular penalty and large margin cosine penalty loss function.
Borui Hou et al., [17] (2021)	Resnet-like architecture with ECA-Resnet blocks trained on arc-cosine loss and softmax loss together.
Weili Yang et al., [18] (2022)	Multi-view finger vein with individual CNNs and view pooling.
Huafeng Qin et al., [19] (2022)	Based on Local Attention Transformer
Tingting Chai et al., [20] (2022)	5 Sequential convolution layers and 1 dense layers
Ismail et al., [21] (2022)	Deeply-fused 3 Sequential convolution layers and 2 dense layers.
Weiye Liu et al., [22] (2023)	Inception architecture with residual attention block.
Zhongxia Zhang et al., [23] (2023)	CNN with spatial and channel attention module.
Chunxin Fang et al., [24] (2023)	Siamese network with attention module.
Chih-Hsien Hsia et al. [25] (2023)	Lightweight Convolutional Neural Network with attention and Margin based loss function.
Yiwei Huang et al. [26] (2023)	Axially enhanced local attention network (criss-cross kernel)
Huang, Junduan et al. [27] (2023)	Frequency spatial coupling network with resnet like architecture
Bin Wa et al., [28] (2023)	3 Sequential convolution layers with bilinear pooling and multiple attention modules.
Xiaoye Li et al., [29] (2023)	Vision Transformer with rigorous regularisation added in mlp.
Raghavendra Ramachandra et al., [30] (2024)	3 Sequential convolution layers followed by multi-head attention module connected in parallel with normal and enhanced finger vein images.
Huafeng Qin et al., [31] (2024)	Generates optimal network using Neural Architecture Search using Gated Recurrent Units with self-attention.
Pengyang Zhao et al. [32] (2024)	Transformer-based view finger vein recognition model which uses enhanced images as attention mask
Enyan Li et al. [33] (2025)	Transformer and convolution-based model processing local and global features.
Proposed Method (2025)	Dynamic Snake Convolution with Graph Convolution layers for feature extraction and novel Centroid-Angular Hybrid Loss.

TABLE 1: Summary of Existing Deep Learning-Based Finger Vein Verification Techniques

including Principal Component Analysis (PCA) and Linear Discriminant Analysis (LDA), have also been applied to improve computational efficiency and reduce feature redundancy. While these approaches have shown strong performance in constrained settings, they often struggle to generalise to variations in image quality, capture devices, and acquisition environments, as discussed in [40].

In recent years, deep learning has been extensively explored for finger vein verification, yielding strong performance gains over traditional methods. A taxonomy of deep learning algorithms for finger vein verification is illustrated in Figure 1, and a summary of representative techniques is provided in Table 1. Early deep learning-based approaches typically employed serial convolutional neural network (CNN) architectures, consisting of a sequence of convolutional layers followed by one or two fully connected

layers. Such models, as used in works like [1], [2], [3], [4], [5], [6], [8], [13], [14], [15], [16], [18], [20], [21], were generally lightweight and computationally efficient; however, they often exhibited limited generalisation capability, particularly when applied to diverse acquisition conditions or unseen data.

Autoencoder-based neural networks have been applied to extract high-level feature representations for finger vein verification [10], [11]. Similarly, deformable filters incorporated into Fully Convolutional Networks (FCNs) have been proposed to improve performance [12]. While these approaches can effectively model spatial variations, they often demand longer training times and exhibit limited generalisability across different imaging sensors. Siamese networks have also gained popularity for finger vein verification [7], [9], [24]. Trained with contrastive loss functions,

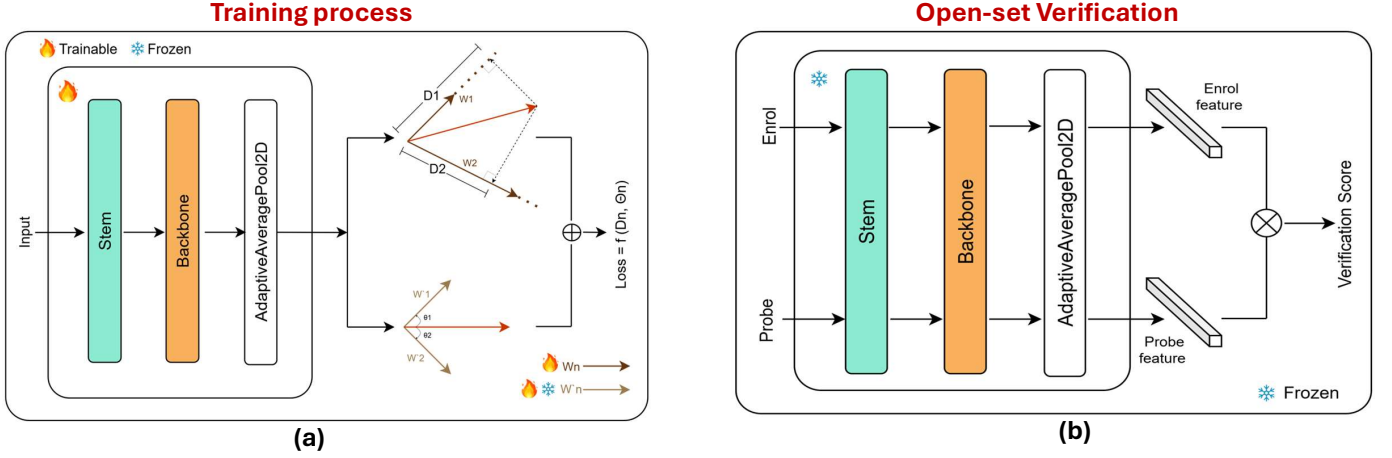


Fig. 2: Block diagram of the proposed OpenVeinNet framework for open-set finger vein verification. (a) The training phase incorporates a STEM module with Dynamic Snake Convolution, followed by a multi-layer Graph Convolutional Network (GCN) and the proposed Graph-Augmented Softmax Loss for robust representation learning. (b) During inference, the system performs open-set verification by comparing probe samples against enrolled templates using the learned graph-based embeddings.

they are relatively robust to class imbalance within datasets. However, these models are computationally intensive to train and can be difficult to scale as the number of enrolled identities increases. Attention-based architectures have attracted growing interest due to their ability to capture both global and local discriminative features in a unified framework [17], [19], [22], [23], [24], [25], [26], [28], [29], [30], [31], [32]. Although these models achieve strong performance on the data they are trained on, their feature embeddings may struggle to generalise to new domains, partly due to their reliance on high-quality training data and susceptibility to overfitting. More recently, Vision Transformer (ViT)-based methods have been explored for finger vein verification [19], [29]. While they show promise, their performance across different sensors is often constrained by the relatively small size of available finger vein datasets, which limits their ability to learn robust, generalisable representations. The latest work from [33] aims to enhance performance by combining multi-scale local and global context features using a dynamic feature interaction network.

Based on the above discussion, one of the most persistent challenges in existing finger vein verification methods is their limited generalisability across datasets captured under varying image qualities and sensing conditions. Most approaches require end-to-end training or dataset-specific fine-tuning to achieve reliable performance, which reduces their adaptability in practical deployments. Furthermore, existing finger vein verification methods often struggle with poor generalisability in open-set scenarios, where training and testing subjects differ, and with limited ability to explicitly model the tubular structure of vein patterns. To address these limitations, we propose a novel architecture, *OpenVeinNet*, designed to enhance both the reliability and generalisability of finger vein verification. The proposed model incorporates Dynamic Snake Convolution (DSC) layers to effectively capture the tubular structures characteristic of finger vein patterns. These features are subsequently processed using Graph Convolutional Neural Networks

(GCNs) to extract highly discriminative representations. For the final verification stage, we employ a lightweight single-layer CNN in a serial configuration with a softmax classifier to compute verification scores. We evaluate the proposed approach on five publicly available finger vein datasets and compare its performance against seven state-of-the-art (SOTA) methods. The results demonstrate that OpenVeinNet consistently outperforms existing techniques in terms of verification accuracy and cross-dataset generalisation. The main contributions of this work are as follows:

- Introduces a novel finger vein verification framework, *OpenVeinNet*, which integrates Dynamic Snake Convolutions with Graph Convolutions in a unified three-stage design to robustly capture and represent discriminative vein features.
- Extensive experimental evaluations on five publicly available datasets such as: FV-300, FV-USM, MM-CBNU, PolyU, and VERA and benchmarking performance against seven state-of-the-art (SOTA) methods.
- Proposes a new task-specific loss function tailored to enhance discriminative feature learning for finger vein verification.
- Provides interpretability analysis of *OpenVeinNet* using Grad-CAM++, offering visual insights into the network's decision-making process.
- Ensures reproducibility by releasing the complete source code and pre-trained models at: <https://github.com/Blazkowicz47/OpenVeinNet>

The remainder of this paper is organised as follows. Section 2 presents the proposed method, detailing its architectural components and the newly introduced loss function. Section 3 reports the quantitative performance of the approach in comparison with state-of-the-art models across multiple datasets. Section 4 outlines the ablation studies conducted to guide the design choices and refine the proposed architecture. Section 5 employs interpretability techniques to visualise and analyse the internal workings of the

model. Finally, Section 6 summarises the key conclusions and highlights potential directions for future research.

2 PROPOSED METHOD: OPENVEINNET FOR OPEN-SET FINGER VEIN VERIFICATION

Finger vein recognition uses the distinctive, curvilinear patterns of veins located beneath the skin for biometric verification. Although these patterns are unique to each finger and individual, the recognition process involves several challenges. These include variations in vein visibility under different imaging conditions, as well as irregular shapes caused by changes in finger pose and rotation during acquisition. As a result, open set verification, where the model must identify individuals who were not seen during training, is not only a technically demanding task but also essential for the practical deployment of such systems. To address these challenges, we propose *OpenVeinNet*, a new framework that combines a Dynamic Snake Convolution Graph Neural Network (DscGrapher) with a Centroid Angular Hybrid Loss. The DscGrapher is designed to model the unique tubular structure of finger veins by making use of Graph Convolutional Networks (GCNs) [41]. It takes the output of the Dynamic Snake Convolution (DSC) and transforms it into graph node embeddings by aggregating features from neighbouring regions. This results in a compact feature representation of the finger vein image, which is then compared using cosine similarity for verification.

Figure 2 shows the block diagram of the proposed *OpenVeinNet* and Table 2 shows the details of the proposed architecture. Part (a) illustrates the training process, while part (b) demonstrates the open set verification phase. During verification, finger vein images are passed through the *OpenVeinNet* to extract discriminative feature embeddings. These embeddings are then compared using cosine distance to assess the similarity between a probe and a reference sample. As shown in Figure 2(a), the *OpenVeinNet* architecture consists of two main components. The first is a stem module that extracts prominent tubular features from the input image. The second is a backbone made up of ten graphers connected in series, which are responsible for capturing both global and local feature representations. In addition to the architectural design, we introduce a new loss function called the Centroid Angular Hybrid Loss. This encourages feature embeddings to remain compact within each class while also ensuring clear separation between different classes. The stem module includes Dynamic Snake Convolution (DSC) layers for adaptive feature extraction, along with a learnable positional embedding that preserves spatial information. The backbone consists of a sequence of Grapher Blocks that are designed to capture long-range connectivity in vein structures, followed by a global pooling layer that generates the final embeddings used for verification. The following sections provide a detailed explanation of each component in the proposed *OpenVeinNet* framework.

2.1 Stem: Dynamic Snake Convolution (DSC) Blocks

Finger veins form complex, non-repeating structures that often resemble branching trees. Standard convolutional layers struggle to capture these patterns effectively because

of their fixed and rigid receptive fields. To overcome this limitation, the Dynamic Snake Convolution (DSConv) adjusts its sampling locations in a flexible manner that follows the natural contours of the vein patterns. This dynamic behaviour allows DSConv to adapt to variations in shape and direction, enabling more accurate extraction of features from faint, curved, or irregular veins.

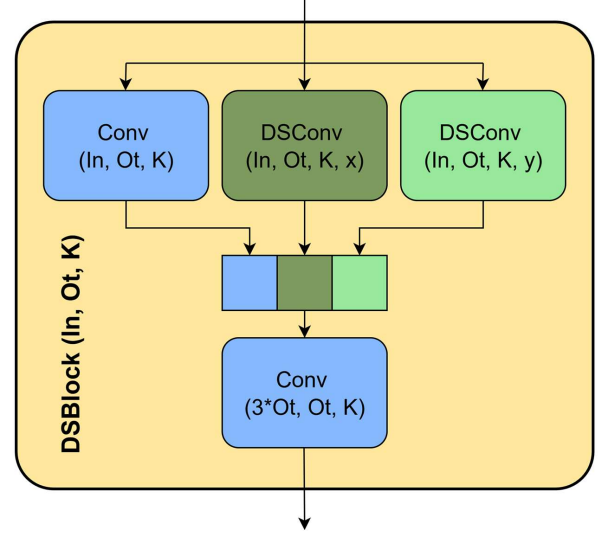


Fig. 3: DSBlock (In: input channels, Ot: output channels, K: kernel size, x and y are the directions in which DSConv operates)

Component	Layer	Output Dimensions
Input	-	224,224,3
Stem	DSC module 1 (S=2, F=3)	(112,112,16)
	DSC module 2 (S=2, F=3)	(56,56,32)
	DSC module 3 (S=1, F=3)	(28,28,64)
Backbone	Stage I (K=9, S=2, F=3)	(13,13,128)
	Stage II (K=9, S=2, F=3)	(6,6,256)
	avg_pool	(1,1,256)

TABLE 2: OpenVeinNet architecture details indicating output dimension. K indicates number of nearest neighbours, S indicates strides, and F indicates filter-size

Figure 3 illustrates the architecture of the proposed DSConv Block. This block incorporates two standard convolution layers and two Dynamic Snake Convolutions (DSCs), which operate independently along the horizontal and vertical directions. This design helps improve learning efficiency and enhances the model’s ability to generalise to complex vein patterns. The input to the DSConv Block is first passed through a regular convolution, followed by two DSCs. The outputs from these three branches are then concatenated and passed through another convolutional layer to produce the final output. Figure 4 shows the architecture of the complete stem module. Each DSConv Block in the stem is followed by a batch normalisation layer and a GELU activation. However, the GELU activation is deliberately omitted after the final DSConv Block. This allows the resulting embeddings to span a wider range of values, which is important for capturing fine-grained vein features.

Given the finger vein image $\mathbf{X} \in \mathbb{R}^{B \times C_{in} \times H \times W}$, where B is the batch size, C_{in} is the number of channels, and $H = 224$, $W = 224$ are the height and width respectively. Each DSConv operates by applying a deformable sampling strategy, based on learned offset maps, to follow vein-like tubular structures in either the x- or y-direction. This process can be described mathematically as follows:

A 3×3 convolution predicts offsets $\mathbf{O} \in \mathbb{R}^{B \times 2K \times H \times W}$, where K is the kernel size (set to 9, 7, or 3 for the three stem layers), and $\mathbf{O} = (\mathbf{O}_y, \mathbf{O}_x)$ contains K offsets for y - and x -directions per pixel:

$$\mathbf{O} = \tanh(\text{GroupNorm}(\text{Conv}_{3 \times 3}(\mathbf{X}))) \quad (1)$$

Let Offsets adjust pixel coordinates be $\mathbf{p} = (p_y, p_x)$, where $\mathbf{p}$ denotes a pixel's position in the spatial grid ($p_y \in \{0, 1, \dots, H-1\}$, $p_x \in \{0, 1, \dots, W-1\}$). The coordinate maps are generated as:

$$\mathbf{Y}_{\text{coord}}, \mathbf{X}_{\text{coord}} = \text{GetCoordinateMap2D}(\mathbf{O}, m, \text{extend_scope}) \quad (2)$$

Where,

- $m \in \{0, 1\}$ specifies x -axis ($m = 0$) or y -axis ($m = 1$) deformation.
- extend_scope is a scalar ($= 1.0$) that scales the offsets ($\mathbf{O}_y \cdot \text{extend_scope}$, $\mathbf{O}_x \cdot \text{extend_scope}$) to control the deformation range.

The coordinates are normalized to $[-1, 1]$ for interpolation.

In the next step, the feature sampled at $\mathbf{p} + \mathbf{O}$ is performed using bilinear interpolation as follows:

$$\mathbf{X}_{\text{deformed}} = \text{GridSample}(\mathbf{X}, [\mathbf{X}_{\text{coord}}, \mathbf{Y}_{\text{coord}}], \text{mode} = \text{"bilinear"}) \quad (3)$$

A 1D convolution processes the deformed features, aligning with vein orientations as follows:

$$\mathbf{X}_{\text{dir}} = \begin{cases} \text{Conv}_{(K,1)}(\mathbf{X}_{\text{deformed}}), & \text{if } m = 0 \\ \text{Conv}_{(1,K)}(\mathbf{X}_{\text{deformed}}), & \text{if } m = 1 \end{cases} \quad (4)$$

In the next step, the output is normalized and activated as follows:

$$\mathbf{X}_{\text{dir}} = \text{ReLU}(\text{GroupNorm}(\mathbf{X}_{\text{dir}})) \quad (5)$$

The feature fusion of outputs from a standard convolution ($\mathbf{X}_{\text{conv}} = \text{Conv}_{K \times K}(\mathbf{X})$), x-axis DSConv ($\mathbf{X}_x = \text{DSConv}_x(\mathbf{X}) = \mathbf{X}_{\text{dir}, m=0}$), and y-axis DSConv ($\mathbf{X}_y = \text{DSConv}_y(\mathbf{X}) = \mathbf{X}_{\text{dir}, m=1}$) are concatenated and processed by a final convolution as follows:

$$\mathbf{X}_{\text{out}} = \text{Conv}_{K \times K}(\text{Concat}(\mathbf{X}_{\text{conv}}, \mathbf{X}_x, \mathbf{X}_y)) \quad (6)$$

Based on the mathematical formulation above, the DSConv optimizes alignment with vein patterns by learning offsets $\mathbf{O}$ as follows:

$$\min_{\mathbf{O}} \sum_{\text{pixels}} \|\mathbf{X}(\mathbf{p} + \mathbf{O}) - \mathbf{V}(\mathbf{p})\|_2^2 \quad (7)$$

where $\mathbf{X}$ is the input feature map, $\mathbf{p}$ is a pixel coordinate, $\mathbf{O}$ is the predicted offset, and $\mathbf{V}(\mathbf{p})$ is the idealized vein pattern at $\mathbf{p}$. The convolutional features ($\mathbf{X}_x, \mathbf{X}_y$) capture vein orientations along the horizontal and vertical directions. When integrated with standard convolutional features, they help the model become more robust to image noise and better at

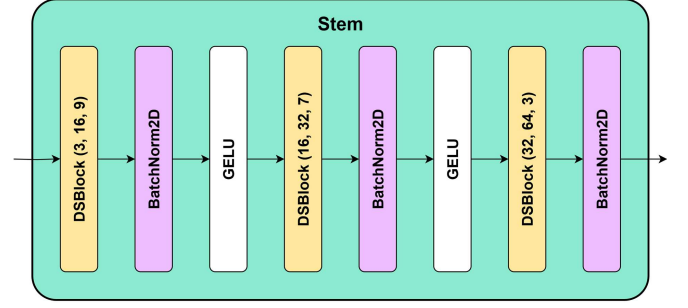


Fig. 4: Stem architecture of OpenVeinNet, featuring a sequence of three Dynamic Snake Convolution (DSConv) layers with kernel sizes $K \in \{9, 7, 3\}$, stride 2, and GELU activation, processing an input image $\mathbf{I} \in \mathbb{R}^{B \times 3 \times 224 \times 224}$ to produce a feature map $\mathbf{X}_{\text{stem}} \in \mathbb{R}^{B \times 64 \times 28 \times 28}$ for the backbone.

detecting subtle vein patterns. This justifies the introduction of DSConv for reliable open-set verification.

Since proposed architecture for stem is having 3 blocks of DSConv, the final output of the stem can be written as:

$$\mathbf{X}_{\text{stem}} = \text{Stem}(\mathbf{I}) \in \mathbb{R}^{B \times C_{\text{out}} \times \frac{H}{2^d} \times \frac{W}{2^d}} \quad (8)$$

Where, $\mathbb{R}^{B \times 64 \times 28 \times 28}$ for $C_{\text{out}} = 64$, $H = W = 224$, using kernel sizes $K \in \{9, 7, 3\}$.

To preserve the spatial arrangement of vein patterns, a learnable positional embedding $\mathbf{P} \in \mathbb{R}^{1 \times C_{\text{out}} \times \frac{H}{2^d} \times \frac{W}{2^d}}$ (e.g., $\mathbb{R}^{1 \times 64 \times 28 \times 28}$) is added to the stem's output as follows:

$$\mathbf{X}_{\text{stem}} = \text{Stem}(\mathbf{I}) + \mathbf{P} \quad (9)$$

Initialized as zeros, $\mathbf{P}$ learns to encode spatial relationships, ensuring the backbone captures the topological structure of veins. The positional embedding enhances feature maps with spatial context as $\mathbf{X}_{\text{stem}}(i, j) \leftarrow \mathbf{X}_{\text{stem}}(i, j) + \mathbf{P}(i, j)$ preserving vein connectivity for the graph-based backbone, which is critical for distinguishing individuals.

2.2 Backbone: Grapher Blocks

The backbone of the OpenVeinNet framework is designed to model the intricate, long-range connectivity of finger vein patterns using Graph Convolutional Neural Networks (GCNs). It processes features extracted by the stem, augmented with positional embeddings, to produce discriminative embeddings for open-set finger vein verification. The backbone, as implemented, consists of two sequential Grapher Blocks, each designed to aggregate features using a k-nearest neighbour (k-NN) graph, followed by an adaptive average pooling layer to produce embeddings, as shown in Figure 6. Each Grapher Block, illustrated in Figure 5, performs initial projection, dynamic graph construction, graph convolution, feed-forward processing, and residual connections to model long-range dependencies. The architecture in paper describes a an extensive structure with four initial Grapher Blocks, a 3×3 convolution with batch normalization, six additional Grapher Blocks, another convolution with batch normalization, and pooling, producing a feature map of $\mathbb{R}^{B \times 256 \times 6 \times 6}$. Here, we focus on the implemented configuration.

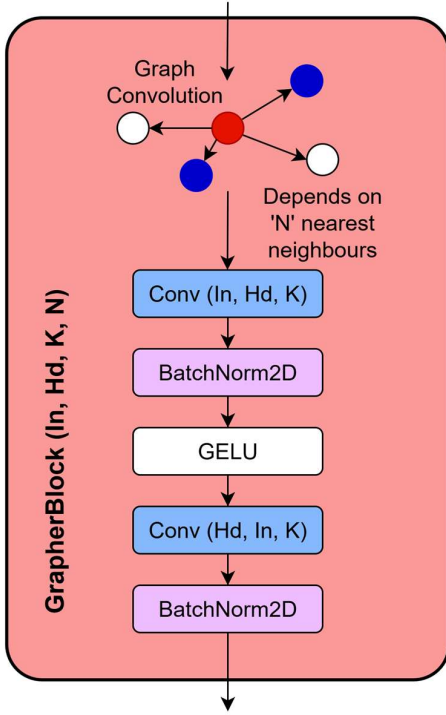


Fig. 5: GrapherBlock (In: input channels, Ot: output channels, K: kernel size, N: nearest neighbours taken into account by graph convolution.)

Given the input $\mathbf{X}_{\text{stem}} \in \mathbb{R}^{B \times 64 \times 28 \times 28}$ from Stem layer, each Grapher Block operates as follows:

- **Initial Projection:** A 1×1 convolution refines the input features, followed by batch normalization:

$$\mathbf{Y} = \text{BatchNorm}(\text{Conv}_{1 \times 1}(\mathbf{X}_{\text{stem}})) \quad (10)$$

- **Dynamic Graph Construction:** A k-NN graph is constructed based on feature similarity, with k neighbors ($k = 18$ for the first block, $k = 9$ for the second):

$$\text{EdgeIndex} = \text{DenseDilatedKnnGraph}(\mathbf{Y}, k, \text{dilation}, \text{stochastic}, \epsilon) \quad (11)$$

where $\text{dilation} = 1$, $\text{stochastic} = \text{False}$, and $\epsilon = 0.2$.

- **Graph Convolution:** Features from neighboring nodes are aggregated using an EdgeConv operation:

$$\mathbf{X}_{\text{out}} = \text{Max}(\text{BasicConv}(\text{Concat}(\mathbf{Y}_i, \mathbf{Y}_j - \mathbf{Y}_i))) \quad (12)$$

where $\mathbf{Y}_i$ and $\mathbf{Y}_j$ are features of nodes i and j , and Max denotes max-pooling over neighbors.

- **Feed-Forward and Residual Connection:** A feed-forward network with two 1×1 convolutions refines features, and a drop-path mechanism ($\text{drop_path} = 0.0$) adds the input to the output.

The graph convolution approximates the maximum similarity between a node and its neighbors:

$$\mathbf{X}_{\text{out}} \approx \max_{j \in \mathcal{N}(i)} \text{Sim}(\mathbf{Y}_i, \mathbf{Y}_j) \quad (13)$$

where $\mathcal{N}(i)$ is the set of k nearest neighbors.

As shown in the Figure 6, the proposed backbone architecture has three main blocks namely: **Block 0:** Input

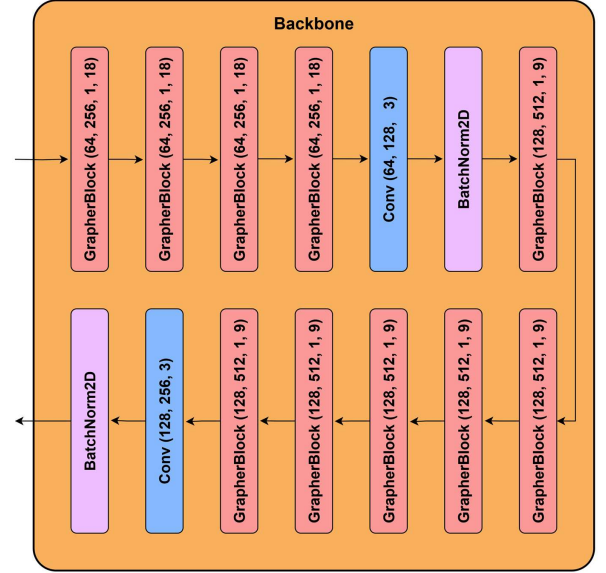


Fig. 6: Backbone structure of OpenVeinNet, with four sequential Grapher Blocks, a convolution layer with batch normalization, six additional Grapher Blocks, another convolution layer with batch normalization, and adaptive average pooling.

channels 64, hidden dimension 256, output channels 128, $k = 18$ neighbors, and a 3×3 shrinker convolution with stride 2, reducing spatial dimensions to 14×14 . This block includes 4 internal Grapher units.

Block 1: Input channels 128, hidden dimension 512, output channels 256, $k = 9$ neighbors, and a 3×3 shrinker convolution with stride 2, reducing spatial dimensions to 7×7 . This block includes 6 internal Grapher units.

Pooling: An adaptive average pooling layer reduces the spatial dimensions to 1×1 :

$$\mathbf{E} = \text{AdaptiveAvgPool2d}(\text{GrapherBlocks}(\mathbf{X}_{\text{stem}})) \in \mathbb{R}^{B \times 256} \quad (14)$$

The feature map transformations yield $\mathbb{R}^{B \times 256 \times 7 \times 7}$ after the second block, which is pooled to produce embeddings $\mathbf{E} \in \mathbb{R}^{B \times 256}$.

The k-nearest neighbour (k-NN) graph construction, using $k = 18$ and $k = 9$ neighbours, captures the topological connectivity of vein patterns. This connectivity is critical for accurate open set verification.

The design of the backbone, illustrated in Figures 6 and 5, builds upon the curvilinear features extracted by the stem and the preserved positional context. Together, these components enable robust feature aggregation, leading to more reliable verification performance.

2.3 Centroid Angular Hybrid Loss

The softmax loss is one of the most widely adopted objectives for classification tasks. It is defined as follows:

$$\mathcal{L}_1 = -\log \frac{e^{W_{y_i}^T x_i + b_{y_i}}}{\sum_{j=1}^N e^{W_j^T x_i + b_j}} \quad (15)$$

Here, $x_i \in \mathbb{R}^d$ represents the embedding of the i -th sample belonging to class y_i . The term $W_j \in \mathbb{R}^d$ denotes the

j -th column of the weight matrix $W \in \mathbb{R}^{d \times N}$, while $b_j \in \mathbb{R}^N$ is the bias term, and N is the total number of classes. In practice, this loss is commonly implemented using a linear layer followed by the `CrossEntropyLoss` module.

While this formulation is effective at training the classifier to favour correct class predictions, it does not explicitly promote intra-class compactness or inter-class separation in the feature space. To address this limitation, we propose a new objective termed the *Centroid Angular Hybrid Loss* ($\mathcal{L}_2$), which enhances the original softmax loss by incorporating angular similarity between the sample and class-specific direction vectors:

$$\mathcal{L}_2 = -\log \frac{e^{W_{y_i}^T x_i + b_{y_i} + \beta \cdot \theta_{y_i}}}{\left(\sum_{j=1}^N e^{W_j^T x_i + b_j}\right) \cdot \left(\sum_{j=1}^N e^{\theta_j}\right)} \quad (16)$$

The angular component θ_j is defined as:

$$\theta_j = \cos^{-1} \left(\frac{\tilde{W}_j^T x_i}{\|\tilde{W}_j\| \cdot \|x_i\|} \right) \quad (17)$$

where $\tilde{W} \in \mathbb{R}^{d \times N}$ has the same shape as W , but is initialised independently. The angular term captures the directional misalignment between the embedding x_i and the centroid-like reference $\tilde{W}_j$, promoting angular discrimination among classes.

The balancing factor β controls the contribution of the angular component. This hybrid formulation encourages not only correct classification but also tighter intra-class clustering and better inter-class separation in angular space, which is particularly useful in open set scenarios where unseen classes may appear during testing.

The proposed Loss function enhances finger vein verification by promoting compact clustering of embeddings for each identity, improving intra-class similarity critical for open-set recognition. Its angular component ensures robust topological alignment, capturing the curvilinear vein patterns effectively.

Figure 2 (a), shows how the weights W target the projection of the feature vector onto the class-weight vectors, while the weights W' target the angle θ between the feature vector and class-weight vector. The weights W' are initially trainable, but we freeze them after 30 epochs. As this term is a destabilising term, during the initial phase of the model, it actively destabilises the model. After the weights become static, they act like a static learned margin. By discouraging the model from fitting the features around W' , we encourage the feature vectors to fit around W more tightly.

2.4 Implementation Details

The proposed *OpenVeinNet* was trained end to end using the AdamW optimiser with a weight decay of 0.05 and an initial learning rate of 1×10^{-3} . Weights were initialised using the Kaiming method with zero bias, while normalisation layers were initialised with weights of 1 and biases of 0. Training was performed for 150 epochs with a mini-batch size of 128 on an NVIDIA L40S GPU. The model contains approximately 4,759,296 (≈ 4.7 million) learnable parameters.

3 EXPERIMENTS AND RESULTS

In this section, we discuss the quantitative results of the proposed and existing finger vein verification methods. The performance is reported using Equal Error Rate (EER) computed at False Match Rate (FMR) = False Non-Match Rate (FNMR) and Area Under Curve (AUC). The performance of the proposed method is compared with seven different SOTA finger vein verification systems that include both handcrafted and deep learning-based methods, such as MCP [40], RLT [40], WLD [40], multiple attention [28], deep fusion [21], self-attention [30], ECA-Resnet with arc-cosine loss [17], vision transformers [29], and local and global feature interaction network [33]. Extensive experiments were performed on five different publicly available finger vein datasets to benchmark the performance of the proposed and SOTA finger vein verification techniques.

3.1 Datasets

The experiments in this work utilise five publicly available finger vein datasets: FV-300 (A) [30], MMCBNU-6000 (B) [42], FV-USM (C) [43], PolyU (D) [44], and VERA (E) [45]. The summary of the publicly available finger vein dataset employed in this work is shown in Table 3. A brief description of each dataset is provided below.

- **FV-300** [30]: Contains 300 distinct finger vein patterns collected from 75 subjects. Images were captured using a custom monochromatic CMOS camera with two lighting sources. For each subject, the index and middle fingers of both hands were imaged across multiple sessions, with intervals ranging from 1 to 4 days. Among the datasets used, FV-300 offers the highest image quality.
- **MMCBNU-6000** [42]: Developed by Chonbuk National University, this dataset includes 600 distinct finger vein patterns from 100 individuals. Acquisition was performed using a camera equipped with an NIR-pass filter and an array of 850 nm infrared LEDs. For each subject, the index, middle, and ring fingers of both hands were captured ten times. The dataset includes participants of diverse nationalities, age groups, blood types, and genders, enabling evaluation of system universality.
- **FV-USM** [43]: Collected at Universiti Sains Malaysia, this dataset comprises 492 distinct finger vein patterns from 123 subjects (83 males and 40 females), aged between 20 and 52. Each subject contributed images of the index and middle fingers of both hands. Data were acquired in two sessions separated by a two-week interval.
- **PolyU** [44]: Contains 312 distinct finger vein patterns from 156 subjects, captured using a peg-free, user-friendly, and unconstrained imaging setup. Data acquisition spanned an 11-month period. In this study, we use version 1 of the database, which consists of 218 unique finger vein patterns, with four images per finger.
- **VERA** [45]: Acquired using an open finger vein sensor, this dataset contains 220 distinct finger vein patterns from 110 subjects. Each subject presented

the index fingers of both hands, with two samples captured per finger at a 5-minute interval. The dataset includes 40 female and 70 male participants aged between 18 and 60, with a mean age of 33 years.

Dataset	Classes	Images	Total Images
FV-300 (A) [30]	300	92*	27,600*
MMCBNU (B) [42]	600	10	6,000
FV-USM (C) [43]	492	12	5,904
POLY-U (D) [44]	218	4	872
VERA (E) [45]	220	2	440

TABLE 3: Describes the data distribution across the train and test split. (*Some images were removed due to bad quality)

3.2 Performance Evaluation Protocol

Dataset	Genuine Scores	Imposter Scores
FV-300 (A) [30]	2,361,300	713,816,582
MMCBNU (B) [42]	54,000	35,940,000
FV-USM (C) [43]	64,944	34,786,368
POLY-U (D) [44]	6214	1,615,588
VERA (E) [45]	440	192,720

TABLE 4: Genuine and imposter score count used for EER calculation

In this study, we adopt an *open-set evaluation protocol* combined with a *leave-one-out* strategy to rigorously assess the performance of deep learning-based models. Specifically, models are trained on four of the five datasets and evaluated on the remaining one, ensuring that the test set contains subjects unseen during training. The embeddings produced by each model’s last layer (prior to the classification layer) are used for evaluation. As illustrated in Figure 2 (a), verification is performed by computing the cosine similarity between the embeddings of the enrolment and probe samples. Table 4 presents the genuine and imposter score distributions for the five datasets used in this work.

To evaluate numerical stability, each training experiment is repeated with multiple random seeds, introducing variability in both weight initialisation and data ordering. Additionally, the samples per identity are deliberately varied between the training and validation sets to assess robustness to distributional shifts. Consistent performance across different seeds and sampling configurations indicates stable convergence under diverse stochastic and data composition conditions. For all reported results, the *mean* Area Under the Curve (AUC) and Equal Error Rate (EER) are computed, along with their corresponding 95% Confidence Intervals (CI), averaged across random seeds. This jointly reflects both model performance and stability under varying initialisation states and sample distributions.

3.3 Results and Discussion

Table 5 presents the performance of the proposed method and several state-of-the-art (SOTA) baselines across five open-set evaluation scenarios. For optimal verification performance, the Area Under the Curve (AUC) should be maximised, while the Equal Error Rate (EER) should be

minimised. The 95% confidence intervals (CI) are reported to reflect the statistical reliability of the results.

Across all open-set cases, the proposed method consistently outperforms both traditional handcrafted feature-based methods (MCP, RLT, WLD) and deep learning-based baselines (ArcVein, LGFIN, FV-ViT, VeinAttNet) in terms of achieving higher AUC and lower EER. The improvement is particularly notable in challenging cross-dataset scenarios where sensor characteristics and image quality differ significantly between training and test sets.

Case BCDE → A: The proposed method achieves an AUC of 99.53% (CI: 99.31–99.74) and an EER of 3.18% (CI: 2.13–4.23), significantly outperforming ArcVein (89.77% AUC, 19.38% EER) and FV-ViT (89.84% AUC, 17.37% EER). The high accuracy here reflects strong generalisation to high-quality datasets (FV-300) despite being excluded from training.

Case ACDE → B: Performance remains strong, with an AUC of 99.17% (CI: 98.89–99.45) and EER of 3.91% (CI: 3.03–4.79), slightly surpassing VeinAttNet (99.16% AUC, 4.26% EER). The narrow margin indicates that both architectures handle MMCBNU data well, though our method offers a measurable stability advantage as shown by the CI.

Case ABDE → C: This is a highly challenging setting due to dataset variability, where many baselines suffer from significant performance degradation (e.g., ArcVein at 88.09% AUC, 19.76% EER). The proposed method achieves a substantial improvement with 97.71% AUC (CI: 96.90–98.53) and 7.53% EER (CI: 5.88–9.18), demonstrating robust cross-sensor generalisation.

Case ABCE → D: The proposed approach attains 98.81% AUC (CI: 98.44–99.18) and 5.26% EER (CI: 4.36–6.16), improving upon VeinAttNet (97.24% AUC, 8.67% EER) by a notable margin. This confirms the model’s ability to handle unconstrained imaging conditions present in the PolyU dataset.

Case ABCD → E: This is the most difficult scenario, as the E dataset is the noisiest and lowest in quality among all datasets, posing an additional challenge for verification performance. Traditional handcrafted methods perform poorly (AUC below 73% for MCP and RLT), with their very high EER further reflecting the impact of noise and degraded image quality. While all deep learning baselines struggle due to the open finger vein sensor’s acquisition variability, the proposed method achieves the highest performance (89.98% AUC, 18.50% EER), outperforming VeinAttNet (87.31% AUC, 21.26% EER). This highlights its relative robustness under extreme cross-domain shifts and low-quality conditions, though the drop in AUC and rise in EER compared to other cases suggests that domain adaptation or augmentation could further improve performance.

Overall Analysis: The proposed method’s superiority can be attributed to two main factors: (i) the Dynamic Snake Convolution layers in the STEM module, which effectively capture the curvilinear vein structures even under noise and illumination changes; and (ii) the graph convolutional backbone, which encodes relational and topological information, enhancing the model’s generalisation in open-set scenarios. The consistently narrow CIs further confirm training stability and reduced sensitivity to random initialisation and data sampling. In summary, the results validate that

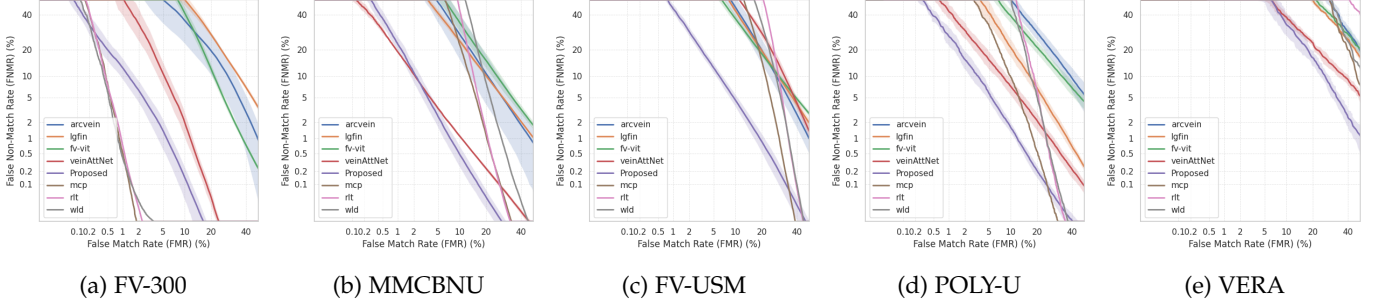


Fig. 7: DET Curves

<i>Train</i> → <i>Test</i> Dataset	Algorithms	AUC (%) (95% CI)	EER (%) (95% CI)
<i>BCDE</i> → <i>A</i>	MCP [34]	99.80	0.89
	RLT [35]	99.78	0.95
	WLD [36]	99.82	0.84
	ArcVein [17]	89.77 (83.13 - 96.42)	19.38 (10.83 - 27.93)
	LGFIN [33]	85.65 (84.32 - 86.99)	23.01 (22.30 - 23.72)
	FV-ViT [29]	89.84 (88.49 - 91.20)	17.37 (15.88 - 18.87)
	VeinAttNet [30]	98.01 (97.14 - 98.87)	6.34 (4.61 - 8.07)
	Proposed Method	99.53 (99.31 - 99.74)	3.18 (2.13 - 4.23)
<i>ACDE</i> → <i>B</i>	MCP [34]	92.90	12.31
	RLT [35]	90.31	13.76
	WLD [36]	87.76	16.98
	ArcVein [17]	91.74 (86.43 - 97.04)	15.47 (9.10 - 21.84)
	LGFIN [33]	92.55 (91.90 - 93.21)	15.12 (14.27 - 15.97)
	FV-ViT [29]	89.75 (87.38 - 92.12)	17.92 (15.47 - 20.38)
	VeinAttNet [30]	99.16 (99.13 - 99.20)	4.26 (4.06 - 4.47)
	Proposed Method	99.17 (98.89 - 99.45)	3.91 (3.03 - 4.79)
<i>ABDE</i> → <i>C</i>	MCP [34]	87.01	18.93
	RLT [35]	79.40	25.14
	WLD [36]	83.07	22.87
	ArcVein [17]	88.09 (84.76 - 91.41)	19.76 (15.95 - 23.58)
	LGFIN [33]	87.74 (85.77 - 89.70)	19.79 (17.73 - 21.85)
	FV-ViT [29]	88.64 (87.30 - 89.98)	19.18 (17.93 - 20.43)
	VeinAttNet [30]	85.94 (84.84 - 87.03)	23.04 (21.45 - 24.63)
	Proposed Method	97.71 (96.90 - 98.53)	7.53 (5.88 - 9.18)
<i>ABCE</i> → <i>D</i>	MCP [34]	95.77	9.90
	RLT [35]	88.17	16.51
	WLD [36]	89.72	15.60
	ArcVein [17]	84.16 (81.09 - 87.23)	23.85 (20.42 - 27.27)
	LGFIN [33]	93.65 (91.84 - 95.45)	13.64 (11.46 - 15.81)
	FV-ViT [29]	87.34 (84.62 - 90.07)	20.58 (18.08 - 23.08)
	VeinAttNet [30]	97.24 (96.35 - 98.13)	8.67 (7.01 - 10.34)
	Proposed Method	98.81 (98.44 - 99.18)	5.26 (4.36 - 6.16)
<i>ABCD</i> → <i>E</i>	MCP [34]	72.21	33.19
	RLT [35]	59.21	44.43
	WLD [36]	69.67	35.45
	ArcVein [17]	69.52 (66.76 - 72.29)	35.57 (33.86 - 37.27)
	LGFIN [33]	74.70 (73.03 - 76.37)	33.16 (31.65 - 34.68)
	FV-ViT [29]	72.23 (69.72 - 74.74)	34.40 (32.11 - 36.69)
	VeinAttNet [30]	87.31 (85.24 - 89.37)	21.26 (19.72 - 22.80)
	Proposed Method	89.98 (86.91 - 93.06)	18.50 (15.94 - 21.06)

TABLE 5: Quantitative performances of various techniques on all the datasets.

the proposed method achieves state-of-the-art performance across diverse open-set conditions, maintaining high accuracy and stability even in the most challenging cross-dataset configurations. Figure 7 presents the Detection Error Trade-off (DET) curves for all evaluated methods across the five open-set verification scenarios. The DET curve illustrates the trade-off between the False Non Match Rate (FNMR) and the False Match Rate (FMR), plotted on a normal deviate scale to better highlight differences in the low-error regions.

4 ABLATION STUDY

In this section, we present and discuss the quantitative results of the study performed to justify the choice of parameters in the proposed OpenVeinNet. We conducted two different refinement studies: 1) to validate the choice of kernel sizes within the DSBlocks of the Stem, and 2) to justify the number of GrapherBlocks employed in the Backbone. In general, existing literature suggests that either a uniform kernel size throughout the network or gradually shrinking the kernel size in deeper layers of the model

Kernel set	EER (in %)
(3, 3, 3)	12.3351
(5, 5, 5)	12.4453
(7, 7, 7)	11.8358
(9, 9, 9)	11.8376
(9, 7, 5)	11.0605
(7, 5, 3)	13.0697
(9, 5, 3)	11.2155
(9, 7, 3)	10.8235

TABLE 6: Set of kernel sizes sequentially in the DSBlocks of the stem

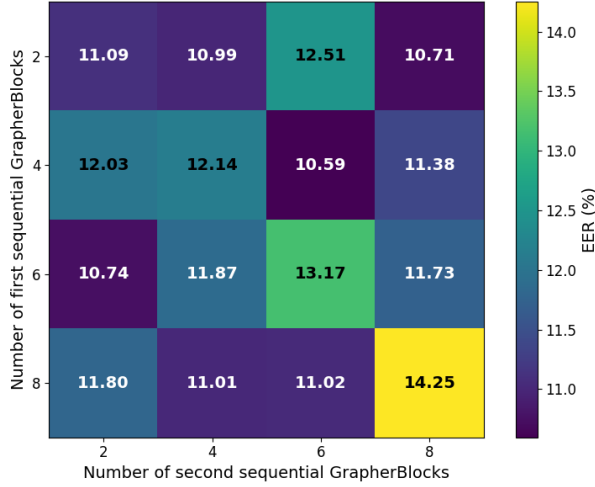


Fig. 8: Experiments refining the number of GrapherBlocks in the Backbone; Y-axis shows number of blocks at the start, X-axis shows them at the end.

yields optimal performance. To identify the most effective configuration for our model while keeping the number of experiments manageable, we chose a specific set of kernel sizes as shown in Table 6.

These results demonstrate that using larger kernels in the shallow layers, followed by progressively smaller kernels, achieves better performance, as indicated by the lowest EER of 10.8235% for the [9, 7, 3] configurations.

The number of sequential GrapherBlocks is proportional to the effective depth of message passing within the GCNs, determining how far information propagates among neighbouring nodes. In order to get the optimal number of GrapherBlocks in the Backbone while simultaneously keeping the experiments limited, we picked four choices for the number of sequential GrapherBlocks at a time to be among: 2, 4, 6, 8. While conducting these experiments, the optimal stem configuration of [9, 7, 3] is chosen.

As the GrapherBlocks at the start and end of Backbone can vary independently, a total of 16 experiments were conducted, as shown in Figure 8. The combination of four GrapherBlocks at the start of the backbone and six GrapherBlocks at the end worked best with an EER of 10.5945%.

In order to assess the effectiveness of the proposed loss function, we trained the proposed network separately with Mean Squared Error (MSE) loss, Negative Log-Likelihood (NLL) loss, and the proposed loss. Performance was evaluated using EER by training using $ABDE \rightarrow C$. As shown

TABLE 7: Study on different loss functions

Loss Function	EER (%)
MSE	26.68
NLL	33.98
Proposed	7.08

in Table 7, the proposed loss consistently outperformed both MSE and NLL, achieves the lowest EER, indicating its superior ability to optimize feature representation for the verification task.

5 INTERPRETATION OF THE PROPOSED METHOD

The accurate interpretation of deep learning models is essential for understanding their decision-making processes, validating their behaviour, and ensuring trust in biometric verification systems. In this work, we adopt Grad-CAM++ [46], an advanced variant of Grad-CAM, to visualise the regions in the input finger vein images that contribute most strongly to the network’s predictions.

To gain a deeper understanding of the proposed model’s internal representations, we conducted two sets of analyses: (i) **genuine comparisons** involving images of the same identity, and (ii) **imposter comparisons** involving images from different identities. Within each set, we examined two cases: maximum similarity between the enrol and probe images, and minimum similarity between them. Grad-CAM++ activations were computed separately for both the stem and backbone components.

As shown in Figure 9 and Figure 10, the Grad-CAM++ activation maps for the stem exhibit strong correspondence to the actual vein patterns in the input images. This indicates that the stem primarily focuses on vascular structures, with vein patterns appearing more sharply in maximum-similarity cases and becoming more diffuse in minimum-similarity cases. In contrast, the backbone activations tend to highlight broader identity-specific regions deeper in the network, suggesting that they encode higher-level discriminative cues beyond the immediate vein texture. These findings confirm the complementary roles of the two components: the stem captures the fine-grained vascular morphology essential for matching, while the backbone refines this representation to emphasise identity-specific features. This separation of responsibilities supports both high verification accuracy and strong generalisation in open-set conditions, thereby reinforcing the reliability and interpretability of the proposed *OpenVeinNet* architecture.

6 CONCLUSION

The inherent security of finger vein biometrics makes them highly suitable for a wide range of secure access control applications. In this work, we introduced **OpenVeinNet**, a novel open-set finger vein verification framework that combines dynamic snake convolutions and graph convolutions for generalised feature extraction and robust verification. The architecture is composed of two main components: a *Stem*, which employs dynamic snake convolutions to capture the tubular vein structures, and a *Backbone*, which leverages graph convolutional layers to encode these structures into discriminative feature representations. The verification

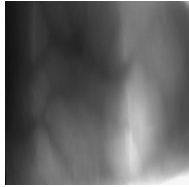
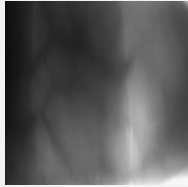
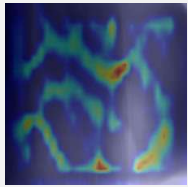
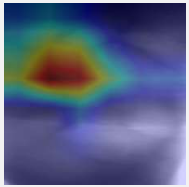
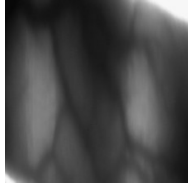
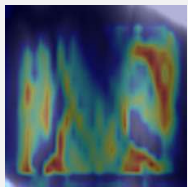
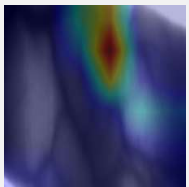
Genuine	Enrol	Probe	GradCam++ (Stem)	GradCam++ (Backbone)
Max Similarity				
Min Similarity				

Fig. 9: Illustration of GradCAM++ [46] based interpretation of the proposed model for fingervein images for the same identity (Genuine comparison).

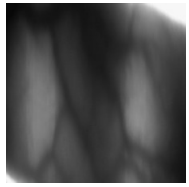
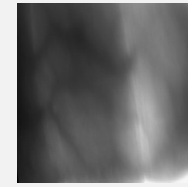
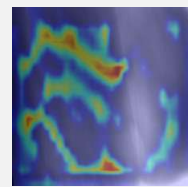
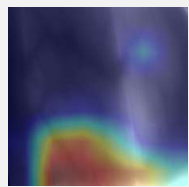
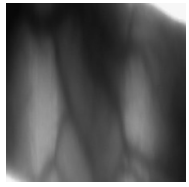
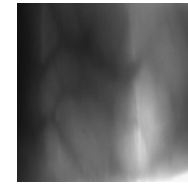
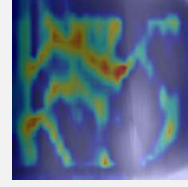
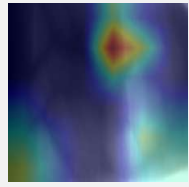
Imposter	Enrol	Probe	GradCam++ (Stem)	GradCam++ (Backbone)
Max Similarity				
Min Similarity				

Fig. 10: Illustration of GradCAM++ [46] based interpretation of the proposed model for fingervein images for the different identity (Imposter comparison).

is performed using the features extracted by the Backbone. To assess its generalisation ability, extensive open-set experiments were conducted on multiple publicly available finger vein datasets. The proposed method was benchmarked against seven state-of-the-art finger vein verification techniques, encompassing both handcrafted and deep learning approaches. Experimental results demonstrate that **OpenVeinNet** consistently achieves superior performance and generalisability across diverse datasets, confirming its effectiveness for secure and reliable biometric verification.

REFERENCES

- [1] H. Qin and M. El Yacoubi, "Finger-vein quality assessment by representation learning from binary images," vol. 9489, 11 2015.
- [2] F. Radzi, S.-I. Khalid, F. Gong, N. Mustafa, Y. C. Wong, and M. Mat Ibrahim, "User identification system based on finger-vein patterns using convolutional neural network," vol. 11, pp. 3316–3319, 03 2016.
- [3] F. Radzi, M. Khalil-Hani, and R. Bakhteri, "Finger-vein biometric identification using convolutional neural network," *TURKISH JOURNAL OF ELECTRICAL ENGINEERING & COMPUTER SCIENCES*, vol. 24, pp. 1863–1878, 01 2016.
- [4] H. Qin and M. A. El-Yacoubi, "Deep representation-based feature extraction and recovering for finger-vein verification," *IEEE Transactions on Information Forensics and Security*, vol. 12, no. 8, pp. 1816–1829, 2017.
- [5] H. G. Hong, M. B. Lee, and K. R. Park, "Convolutional neural network-based finger-vein recognition using nir image sensors," *Sensors*, vol. 17, no. 6, 2017. [Online]. Available: <https://www.mdpi.com/1424-8220/17/6/1297>
- [6] R. Das, E. Piciucchi, E. Maiorana, and P. Campisi, "Convolutional neural network for finger-vein-based biometric identification,"

- IEEE Transactions on Information Forensics and Security*, vol. 14, no. 2, pp. 360–373, 2019.
- [7] C. Xie and A. Kumar, “Finger vein identification using convolutional neural network and supervised discrete hashing,” *Pattern Recognition Letters*, vol. 119, pp. 148–156, 2019, deep Learning for Pattern Recognition. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0167865517304397>
 - [8] J. M. Song, W. Kim, and K. R. Park, “Finger-vein recognition based on deep densenet using composite image,” *IEEE Access*, vol. 7, pp. 66 845–66 863, 2019.
 - [9] S. Tang, S. Zhou, W. Kang, Q. Wu, and F. Deng, “Finger vein verification using a siamese cnn,” *IET Biometrics*, vol. 8, no. 5, pp. 306–315, 2019. [Online]. Available: <https://ietresearch.onlinelibrary.wiley.com/doi/abs/10.1049/iet-bmt.2018.5245>
 - [10] B. Hou and R. Yan, “Convolutional autoencoder model for finger-vein verification,” *IEEE Transactions on Instrumentation and Measurement*, vol. 69, no. 5, pp. 2067–2074, 2020.
 - [11] W. Yang, C. Hui, Z. Chen, J.-H. Xue, and Q. Liao, “Fv-gan: Finger vein representation using generative adversarial networks,” *IEEE Transactions on Information Forensics and Security*, vol. 14, no. 9, pp. 2512–2524, 2019.
 - [12] J. Zeng, F. Wang, J. Deng, C. Qin, Y. Zhai, J. Gan, and V. Piuri, “Finger vein verification algorithm based on fully convolutional neural network and conditional random field,” *IEEE Access*, vol. 8, pp. 65 402–65 419, 2020.
 - [13] R. S. Kuzu, E. Piciucco, E. Maiorana, and P. Campisi, “On-the-fly finger-vein-based biometric recognition using deep neural networks,” *IEEE Transactions on Information Forensics and Security*, vol. 15, pp. 2641–2654, 2020.
 - [14] Z. Hao, P. Fang, and H. Yang, “Finger vein recognition based on multi-task learning,” in *Proceedings of the 2020 5th International Conference on Mathematics and Artificial Intelligence*, ser. ICAI ’20. New York, NY, USA: Association for Computing Machinery, 2020, p. 133–140. [Online]. Available: <https://doi.org/10.1145/3395260.3395277>
 - [15] H. Ren, L. Sun, J. Guo, C. Han, and F. Wu, “Finger vein recognition system with template protection based on convolutional neural network,” *Knowledge-Based Systems*, vol. 227, p. 107159, 2021. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0950705121004226>
 - [16] R. Salih Kuzu, E. Maiorana, and P. Campisi, “Loss functions for cnn-based biometric vein recognition,” in *2020 28th European Signal Processing Conference (EUSIPCO)*, 2021, pp. 750–754.
 - [17] B. Hou and R. Yan, “Arcvein-arccosine center loss for finger vein verification,” *IEEE Transactions on Instrumentation and Measurement*, vol. 70, pp. 1–11, 2021.
 - [18] W. Yang, J. Huang, Z. Chen, J. Zhao, and W. Kang, “Multi-view finger vein recognition using attention-based mvncnn,” in *Biometric Recognition*, W. Deng, J. Feng, D. Huang, M. Kan, Z. Sun, F. Zheng, W. Wang, and Z. He, Eds. Cham: Springer Nature Switzerland, 2022, pp. 82–91.
 - [19] H. Qin, R. Hu, M. A. El-Yacoubi, Y. Li, and X. Gao, “Local attention transformer-based full-view finger-vein identification,” *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 33, no. 6, pp. 2767–2782, 2023.
 - [20] T. Chai, J. Li, S. Prasad, Q. Lu, and Z. Zhang, “Shape-driven lightweight cnn for finger-vein biometrics,” *Journal of Information Security and Applications*, vol. 67, p. 103211, 2022. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S2214212622000886>
 - [21] I. Boucherit, M. O. Zmirli, H. Hentabli, and B. A. Rosdi, “Finger vein identification using deeply-fused convolutional neural network,” *Journal of King Saud University - Computer and Information Sciences*, vol. 34, no. 3, pp. 646–656, 2022. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1319157820303372>
 - [22] W. Liu, H. Lu, Y. Wang, Y. Li, Z. Qu, and Y. Li, “Mmran: A novel model for finger vein recognition based on a residual attention mechanism,” *Applied Intelligence*, vol. 53, no. 3, pp. 3273–3290, Feb 2023. [Online]. Available: <https://doi.org/10.1007/s10489-022-03645-7>
 - [23] Z. Zhang and M. Wang, “Finger vein recognition based on lightweight convolutional attention model,” *IET Image Processing*, vol. 17, no. 6, pp. 1864–1873, 2023. [Online]. Available: <https://ietresearch.onlinelibrary.wiley.com/doi/abs/10.1049/ipr2.12761>
 - [24] C. Fang, H. Ma, and J. Li, “A finger vein authentication method based on the lightweight siamese network with the self-attention mechanism,” *Infrared Physics & Technology*, vol. 128, p. 104483, 2023. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1350449522004649>
 - [25] C.-H. Hsia, L.-Y. Ke, and S.-T. Chen, “Improved lightweight convolutional neural network for finger vein recognition system,” *Bioengineering*, vol. 10, no. 8, 2023. [Online]. Available: <https://www.mdpi.com/2306-5354/10/8/919>
 - [26] Y. Huang, H. Ma, and M. Wang, “Axially enhanced local attention network for finger vein recognition,” *IEEE Transactions on Instrumentation and Measurement*, vol. 72, pp. 1–10, 2023.
 - [27] J. Huang, A. Zheng, M. S. Shakeel, W. Yang, and W. Kang, “Fvf-net: Frequency-spatial coupling network for finger vein authentication,” *IEEE Transactions on Information Forensics and Security*, vol. 18, pp. 1322–1334, 2023.
 - [28] B. Ma, K. Wang, and Y. Hu, “Finger vein recognition based on bilinear fusion of multiscale features,” *Scientific Reports*, vol. 13, no. 1, p. 249, Jan. 2023.
 - [29] X. Li and B.-B. Zhang, “Fv-vit: Vision transformer for finger vein recognition,” *IEEE Access*, vol. 11, pp. 75 451–75 461, 2023.
 - [30] R. Ramachandra and S. Venkatesh, “Fingervein verification using convolutional multi-head attention network,” in *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, January 2024, pp. 6175–6184.
 - [31] H. Qin, C. Fan, S. Deng, Y. Li, M. A. El-Yacoubi, and G. Zhou, “Ag-nas: An attention gru-based neural architecture search for finger-vein recognition,” *IEEE Transactions on Information Forensics and Security*, vol. 19, pp. 1699–1713, 2024.
 - [32] P. Zhao, Y. Song, S. Wang, J.-H. Xue, S. Zhao, Q. Liao, and W. Yang, “Vpformer: A transformer-based multi-view finger vein recognition model and a new benchmark,” *Pattern Recognition*, vol. 148, p. 110170, 2024. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0031320323008671>
 - [33] E. Li, L. Yang, K. Su, and H. Liu, “Local and global feature interaction network for partial finger vein recognition,” *IEEE Signal Processing Letters*, vol. 32, pp. 906–910, 2025. [Online]. Available: <https://api.semanticscholar.org/CorpusID:276369489>
 - [34] N. Miura, A. Nagasaka, and T. Miyatake, “Extraction of finger-vein patterns using maximum curvature points in image profiles,” *IEICE Trans. Inf. Syst.*, vol. 90-D, pp. 1185–1194, 2007. [Online]. Available: <https://api.semanticscholar.org/CorpusID:16384087>
 - [35] N. Miura, A. Nagasaka, and T. Miyatake, “Feature extraction of finger-vein patterns based on repeated line tracking and its application to personal identification,” *Machine Vision and Applications*, vol. 15, pp. 194–203, 10 2004.
 - [36] B. Huang, Y. Dai, R. Li, D. Tang, and W. Li, “Finger-vein authentication based on wide line detector and pattern normalization,” in *2010 20th International Conference on Pattern Recognition*, 2010, pp. 1269–1272.
 - [37] K. Wang, A. S. Khisa, X. Wu, and Q. Zhao, “Finger vein recognition using lbp variance with global matching,” in *International Conference on Wavelet Analysis and Pattern Recognition*, July 2012, pp. 196–201.
 - [38] B. A. Rosdi, C. W. Shing, and S. A. Suandi, “Finger vein recognition using local line binary pattern,” *Sensors*, vol. 11, no. 12, pp. 11 357–11 371, 2011.
 - [39] Y. Lu, S. Yoon, S. J. Xie, J. Yang, Z. Wang, and D. S. Park, “Finger vein recognition using histogram of competitive gabor responses,” in *22nd International Conference on Pattern Recognition*, Aug 2014, pp. 1758–1763.
 - [40] R. Raghavendra, J. Surbiryala, and C. Busch, “Hand dorsal vein recognition: Sensor, algorithms and evaluation,” in *2015 IEEE International Conference on Imaging Systems and Techniques (IST)*, 2015, pp. 1–6.
 - [41] G. Li, M. Müller, A. Thabet, and B. Ghanem, “Deepgcns: Can gcns go as deep as cnns?” in *The IEEE International Conference on Computer Vision (ICCV)*, 2019.
 - [42] Y. Lu, S. Xie, S. Yoon, Z. Wang, and D. Park, “An available database for the research of finger vein recognition,” vol. 1, 12 2013, pp. 410–415.
 - [43] M. S. Mohd Asaari, S. A. Suandi, and B. A. Rosdi, “Fusion of band limited phase only correlation and width centroid contour distance for finger based biometrics,” *Expert Systems with Applications*, vol. 41, no. 7, pp. 3367–3382, 2014. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0957417413009536>
 - [44] A. Kumar and Y. Zhou, “Human identification using finger im-

- ages," *IEEE Transactions on Image Processing*, vol. 21, no. 4, pp. 2228–2244, 2012.
- [45] P. Tome, R. Raghavendra, C. Busch, S. Tirunagari, N. Poh, B. H. Shekar, D. Gragnaniello, C. Sansone, L. Verdoliva, and S. Marcel, "The 1st competition on counter measures to finger vein spoofing attacks," in *2015 International Conference on Biometrics (ICB)*, 2015, pp. 513–518.
 - [46] A. Chattopadhyay, A. Sarkar, P. Howlader, and V. N. Balasubramanian, "Grad-cam++: Generalized gradient-based visual explanations for deep convolutional networks," in *2018 IEEE Winter Conference on Applications of Computer Vision (WACV)*, 2018, pp. 839–847.